TRENTOS-M I2C (Chaser Light) Demo for the BD-SL-i.MX6

General

This demo application drives a chaser light setup over I2C via a port expander (MCP23017). The demo currently only works for the.

Modifications to *libplatsupport*

The problem of the I2C demo was the referenced i2c libplatsupport files for the BD-SL-i.MX6 under *seos_sandbox/sdk-sel4-camkes/libs/sel4_util_libs /libplatsupport/src/plat/imx6/i2c.c*. In the file *i2c.c* Data61 makes some modifications to the address with *I2CDATA_WRITE()*, that I do not understand currently and that prevent the I2C demo from working properly. According to the TRM of the port expander (MCP23017): https://ww1.microchip.com /downloads/en/devicedoc/20001952c.pdf though, the target address is "0100 A2 A1 A0 R/W". As all address lines of the chip were set to 0 externally on the breadboard (A1=0,A2=0,A3=0) and we wanted a write access (R/W=0), this meant: setting 0x40 manually in *master_start()*. A smarter way to implement this would be to use

write operation

```
slave = (0x40 | slave);master_start(dev, slave);
```

in *master_txstart()* or

read operation

```
slave = (0x41 | slave);master_start(dev, slave);
```

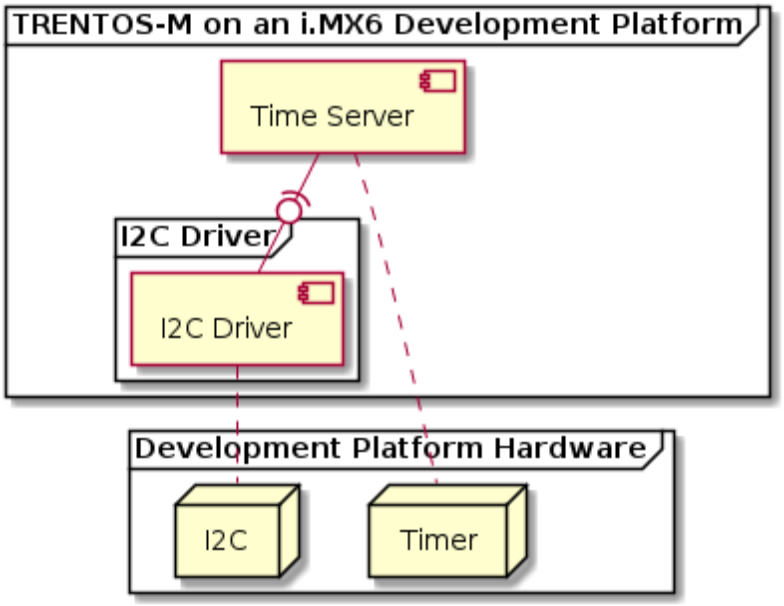
in *master_rxstart()*.

As this modification is specific to the demo and does not have any benefit for external users, there is no sense in upstreaming this change to Data61. This resulted in extracting this file and corresponding include files to the I2C component in the I2C demo on the TRENTOS Software Layer. For future endeavors, the I2C component should be extracted from the application itself. This would mean to create a more general I2C component for multiple platforms.

As the demo now runs, this proves that I2C write operations is generally working for the BD-SL-i.MX6. There is still the need to test if I2C read operations also work (another demo?).

There is still an issue with I2C write operations sometimes failing (maybe it has more to do with the fact that communication sometimes fails).

CAmkES Component Architecture



I2C Running Light Demo for i.MX6-CAmkES Architecture

Hardware Setup

This demo is supposed to run standalone on the BD-SL-i.MX6.

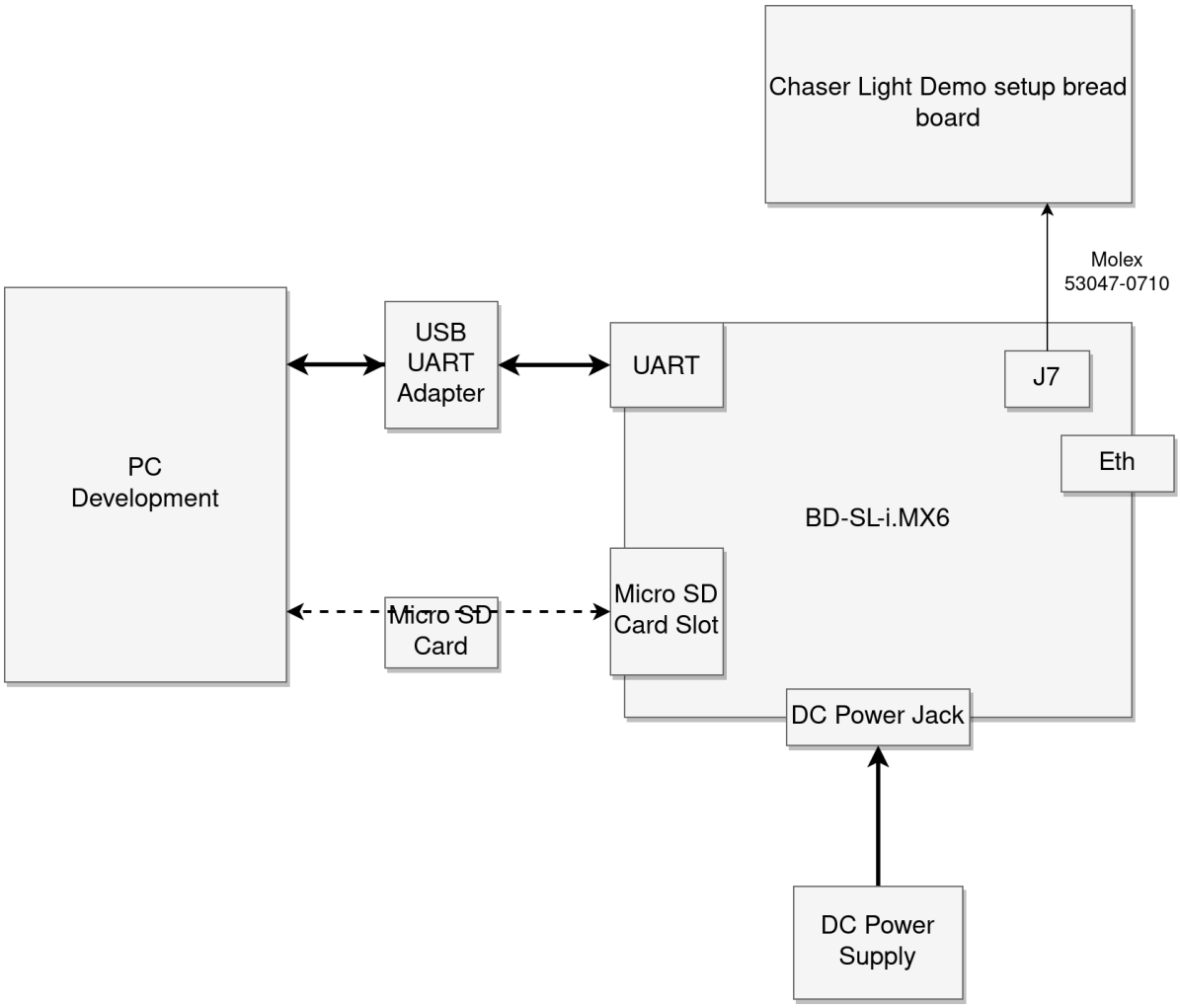
In order to run the demo, the following is required:

- A BD-SL-i.MX6 development board incl. power supply
- USB-to-RS232 cable for console and logs
- A micro SD-Card
- A Molex 53047-0710 connector cable
- The chaser light demo setup bread board

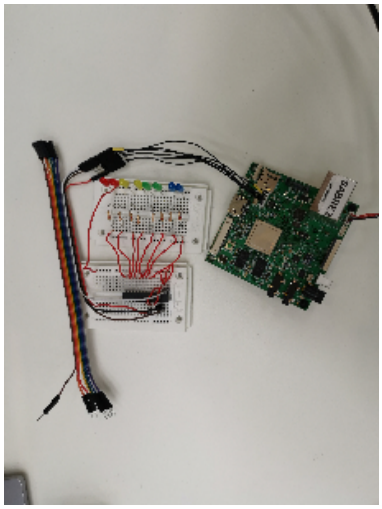
Before building or executing the demo it is necessary to first properly connect all the hardware components:

- connect the USB-to-RS232 cable to the board as described in more detail in the platform support chapter (for the BD-SL-i.MX6).
- connect the USB-to-RS232 adapter to your PC.
- Connect the Molex 53047-0710 connector cable to the J7 pins.
- Connect the Molex 53047-0710 connector cable to the bread board.
- connect the development board to the power supply.

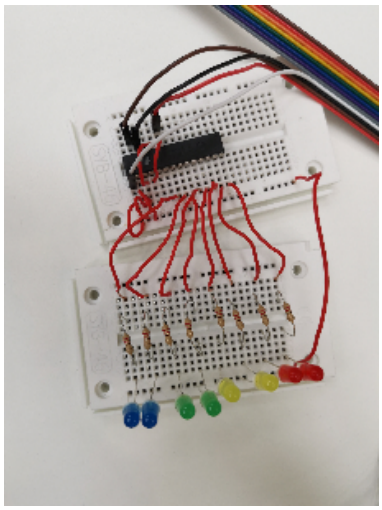
The following diagram shows the hardware setup:



I2C Chaser Light Demo whole setup



I2C Chaser Light Demo bread board setup



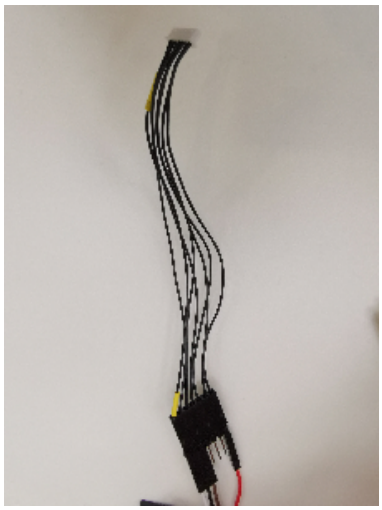
The expansion board setup is based on Chapter 16 of the *Franzis Raspberry Pi Maker Kit*. The following components are required in order to run the *demo_i2c* application:

- 2 x bread boards
- 8 x 220 Ohm resistors (red-red-brown)
- 8 x LEDs
- 4 x connection cable (to connect to Sabre Lite)
- 14 x cables for expansion board itself
- 1 x Molex 53047-0710 connector cable
- 1 x MCP23017 port expander

Molex to Sabre Lite connection

The molex connector end is inserted in the J7 connector slot on the BD-SL-i.MX6. Thus, the chaser light demo uses the I2C3 of the board.

Molex 53047-0710 connector cable



The data lines of the J7 connector are then connected to the port expander according to the following table:

J7-PIN#	J7-Function	MCP23017-PIN#	MCP23017-Term
1	+5V	VDD	9
5	I2C3_SDA	SDA	13
6	I2C3_SCL	SCK	12
7	GND	A0	15

Building the demo

For building the I2C (chaser light) demo, the **`build-system.sh`** script has to be used and executed within the **trentos_build** docker container. The following command will invoke this custom build script from inside the **trentos_build** docker container. The container will bind the current working folder to a volume mounted under **`/host`** , execute the script and then self remove.

IoT Demo for i.MX6 - Call build-system.sh in the Container Environment

```
# Entering the SDK root directorycd <sdk_root_directory> # Building the demo for the BD-SL-i.MX6sdk/scripts
/open_trentos_build_env.sh \    sdk/build-system.sh \    sdk/demos/demo_i2c/src \    sabre \    build-Debug-
demo_i2c \    -DCMAKE_BUILD_TYPE=Debug
```

As a result, the folder **`build-Debug-demo_i2c`** is created, containing all the build artifacts.

The TRENTOS-M system image is **`build-Debug-demo_i2c/images/os_image.bin`**.

For an in-depth discussion about building TRENTOS-M systems, different possible configurations and parameters, please refer to the TRENTOS Buildsystem section.

Running the demo

To boot the board from the SD card, a suitable boot script has to be placed on the SD card. The boot files for the used platform can be found in **`sdk/resources`** in **`sdk/resources/sabre_sd_card/`** (BD-SL-i.MX6).

I2C demo for i.MX6 (BD-SL-i.MX6)

```
# copy BD-SL-i.MX6 bootfiles to SD Card BOOT partitioncp sdk/resources/sabre_sd_card/*
<sd_card_mount_point_BOOT>/
```

The previously built TRENTOS-M system image needs to be placed on the SD card along with the boot script. The system image **`os_image.bin`** has been created in the **`build-Debug-demo_i2c/images/`**.

I2C demo for i.MX6 (BD-SL-i.MX6)

```
# copy TRENTOS-M system image (the I2C demo application) to SD Card BOOT paritioncp build-Debug-demo_i2c
/images/os_image.bin <sd_card_mount_point>/  # ensure files are written to the SD Cardsyncumount
<sd_card_mount_point>
```

Start a serial monitor that shows the traffic received from the UART-to-USB adapter. One way of doing this is to use the **`picocom`** utility with the following command.

I2C demo for i.MX6 - Starting the Serial Monitor

```
sudo picocom -b 115200 /dev/<ttyUSBX>
```

Hereby, **`<ttyUSBX>`** acts as a placeholder for the specific device representing the USB-to-UART adapter, e.g. **`ttyUSB0`**. Note that using **`sudo`** may not be required, this depends on your Linux group membership giving your account access to **`/dev/<ttyUSBX>`**.

Place the prepared SD card in the board and power it up. After starting up, you should be able to see a similar result as in the following video: